

Sujet d'examen UE NFE204 - Session 1

Année universitaire 2018-2019

Examen première session : 29 janvier 2019

Consignes

Durée de l'examen : 3h00 ; Documents non autorisés ; Calculatrice autorisée

Les exercices sont indépendants les uns des autres. Merci de soigner votre écriture.

Ce sujet contient 3 pages.

Important : nous n'évaluons pas les réponses par la syntaxe. Ne vous inquiétez pas d'utiliser un point-virgule à la place d'un point d'exclamation ou autres détails mineurs

L'ensemble de l'examen doit vous permettre de montrer que vous avez assimilé les principales notions vues en cours et en TP et que vous savez les mettre en application et les expliquer. La clarté, la concision, la précision des réponses sera donc appréciée.

Première partie : documents structurés et MapReduce (7 pts)

Nous gérons un site de commerce électronique qui recense des restaurants et permet à des internautes de les noter. Pour l'instant nous utilisons un système relationnel, dont voici le schéma (très simplifié).

- Restaurant (**id**, nom)
- Internaute (**id**, nom)
- Notation (**idRestau**, **idInternaute**, note, commentaire)

Voici également un tout petit échantillon de la base.

id	nom	id	nom	idRestau	idInternaute	note	commentaire
20	Aux délices	100	Marie	20	100	1	Nul
21	Le régál	101	Adrien	20	101	5	Parfait
				21	101	3	Moyen

La table *Restaurant* La table *Internaute* La table *Notation*

On veut implanter un algorithme de filtrage collaboratif. Un des calculs consiste notamment à déterminer la note moyenne donnée par chaque internaute. Quelqu'un décide alors de remplacer la base relationnelle par une base NoSQL, et SQL par MapReduce. Voyons s'il a raison.

1. Donnez une représentation de l'échantillon ci-dessus sous la forme d'une collection de documents JSON, centrée sur les entités "Restaurant". Nous l'appellerons représentation A.
2. Proposez une autre représentation, centrée sur les entités "Internaute". Nous l'appellerons représentation B.
3. Donnez la requête SQL qui calcule la note moyenne donnée par chaque internaute.
4. Expliquez le calcul MapReduce donnant cette moyenne, pour la représentation A.
5. Même chose pour la représentation B.
6. La phase de Reduce peut-elle être évitée pour l'un des calculs MapReduce ? Si oui expliquez pourquoi, et l'impact sur la performance du calcul.

Important : pour spécifier les calculs MapReduce, donner les fonctions de Map et de Reduce, en javascript, en pseudo-code, au pire en langage naturel en étant le plus précis possible.

Deuxième partie : recherche d'information (8 pts)

On ajoute dans les documents des commentaires sur les restaurants, désignés par $c_1, \dots, c_5$. On veut maintenant créer un index plein texte pour rechercher des commentaires. On va travailler sur l'échantillon suivant.

- c_1 Plats médiocres, bonne carte des vins, accueil cordial.
- c_2 Le vin est moyen, les desserts sont mauvais, mais le décor est agréable.
- c_3 En général, le vin est trop cher au restaurant, mais ici la carte des vins est complète et de qualité.
- c_4 Passons sur le décor d'un goût douteux. Les plats et desserts sont délicieux, c'est tout ce qui compte après tout.
- c_5 Plat, dessert et un verre de vin pour un prix correct. Le plat du jour est toujours intéressant.

On va se limiter au vocabulaire (vin, plat, décor, dessert). (NB : on suppose une phase préalable de normalisation qui élimine les pluriels, majuscules, etc.)

- Donnez une matrice d'incidence contenant les tf, et un tableau donnant les idf (sans le log), pour les termes précédents. Placez les termes en ligne, les commentaires en colonne.

Solution :

	c1	c2	c3	c4	c5
vin 5/4	1	1	2	0	1
plat 5/3	1	0	0	1	2
décor 5/2	0	1	0	1	0
dessert 5/3	0	1	0	1	1

- Donnez les normes des vecteurs représentant ces commentaires.

Solution : Les normes

$$\|c_1\| = \sqrt{1+1}; \|c_2\| = \sqrt{3}; \|c_3\| = \sqrt{4}; \|c_4\| = \sqrt{3}; \|c_5\| = \sqrt{6}$$

- Donner les résultats classés par similarité cosinus basée sur les tf (on ignore l'idf) pour les requêtes suivantes.

- plat;
- décor et dessert;
- vin et plat.

Expliquez brièvement la raison du classement pour le premier document.

Solution : Les cosinus (requête non normalisée).

$$\text{— plat : } c_1 : \frac{1}{\sqrt{2}}; c_4 : \frac{1}{\sqrt{3}}; c_5 : \frac{2}{\sqrt{6}};$$

— décor et dessert :

$$c_2 : \frac{1+1}{\sqrt{3}}; c_4 : \frac{1+1}{\sqrt{3}}; c_5 : \frac{1}{\sqrt{6}};$$

— vin et plat

$$c_1 : \frac{1+1}{\sqrt{2}}; c_2 : \frac{1}{\sqrt{3}}; c_3 : \frac{2}{\sqrt{3}}; c_4 : \frac{1}{\sqrt{3}}; c_5 : \frac{1+2}{\sqrt{6}};$$

- Pour la requête "décor et dessert", et les documents c2 et c4, qu'est-ce qui change si on ne met pas de restriction sur le vocabulaire (tous les mots sont indexés) ?

Solution : Dans ce cas le plus petit document (c2) l'emporte car sa norme est plus petite. Intuitivement : il parle plus *spécifiquement* de décor et de dessert que c4.

5. On voudrait pouvoir mesurer les co-occurrences de termes, autrement dit produire un indicateur représentant la probabilité que deux termes apparaissent ensembles dans un même commentaire. Proposez une formule estimant cette probabilité et s'appuyant sur les informations présentes dans la matrice d'incidence.

Troisième partie : flux (2 pts)

Nous recevons un flux de notes données à des restaurants nommé `restaus`. Chaque élément contient le nom du restaurant, l'identifiant de l'utilisateur, la note et le commentaire. Voici un échantillon.

```
1,Le bougnat,100,4,"Plats médiocres ..."  
2,Délices de Saïgon,100,2,"Desserts nuls..."  
3,Le régal,101,4,"venez pour le décor"  
..
```

On met en place une chaîne de traitement pour ce flux. Voici cette chaîne exprimée avec les opérateurs de Pig

```
coll1 = filter restaus by contains(commentaire, "dessert")  
coll2 = group coll1 by nomRestaurant;  
coll3 = foreach coll2 generate group, AVG(coll1.note);
```

1. Que produit cette chaîne ?

Solution : La note moyenne, par restaurant, des commentaires relatifs aux desserts

2. Quel est le degré maximal de parallélisation ?

Solution : La parallélisation est limitée par le nombre de restaurants.

Quatrième partie : questions de cours (3 pts)

1. Je soumetts une requête $t_1, t_2, \dots, t_n$. Quel est le poids de chaque terme dans le vecteur représentant cette requête ? La normalisation de ce vecteur est elle importante pour le classement (justifier) ?
2. Dans un traitement MapReduce, à quoi correspond l'opération dite de *shuffle* ?
3. Dans un système distribué avec partitionnement, je décide de partitionner ma collection de films sur le genre. Discuter des avantages et inconvénients de ce choix.